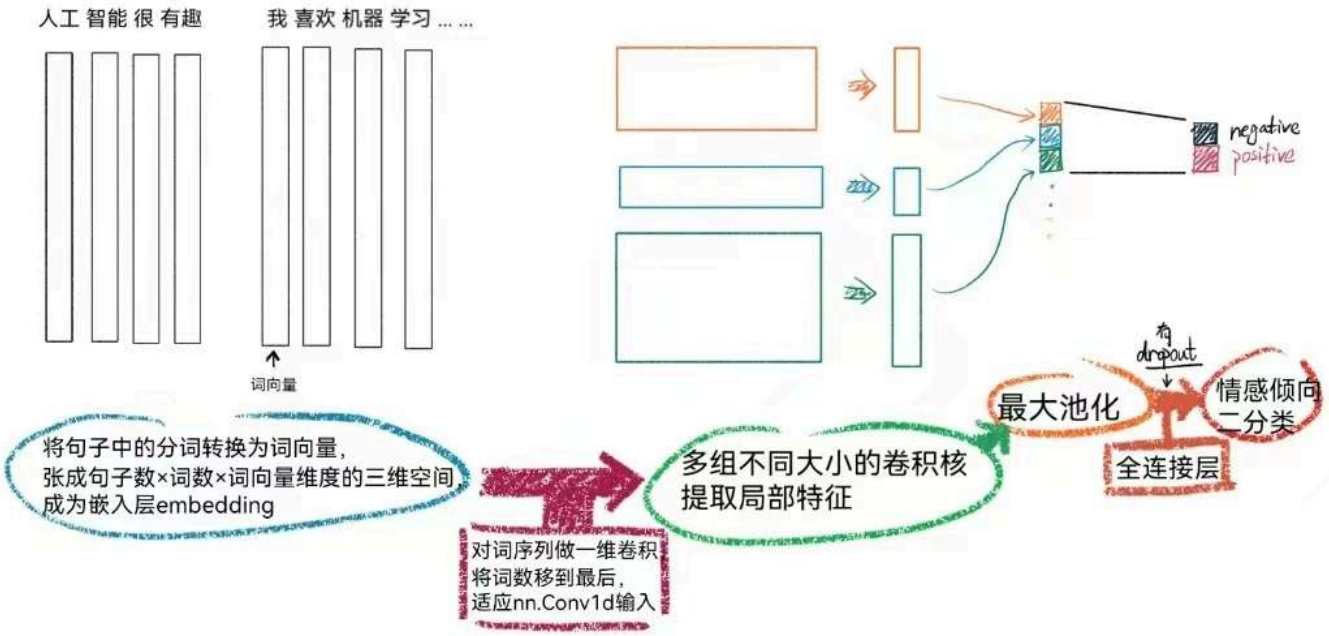
代码网盘链接

```
https://cloud.tsinghua.edu.cn/d/69baaa2669be47fdafca/
```

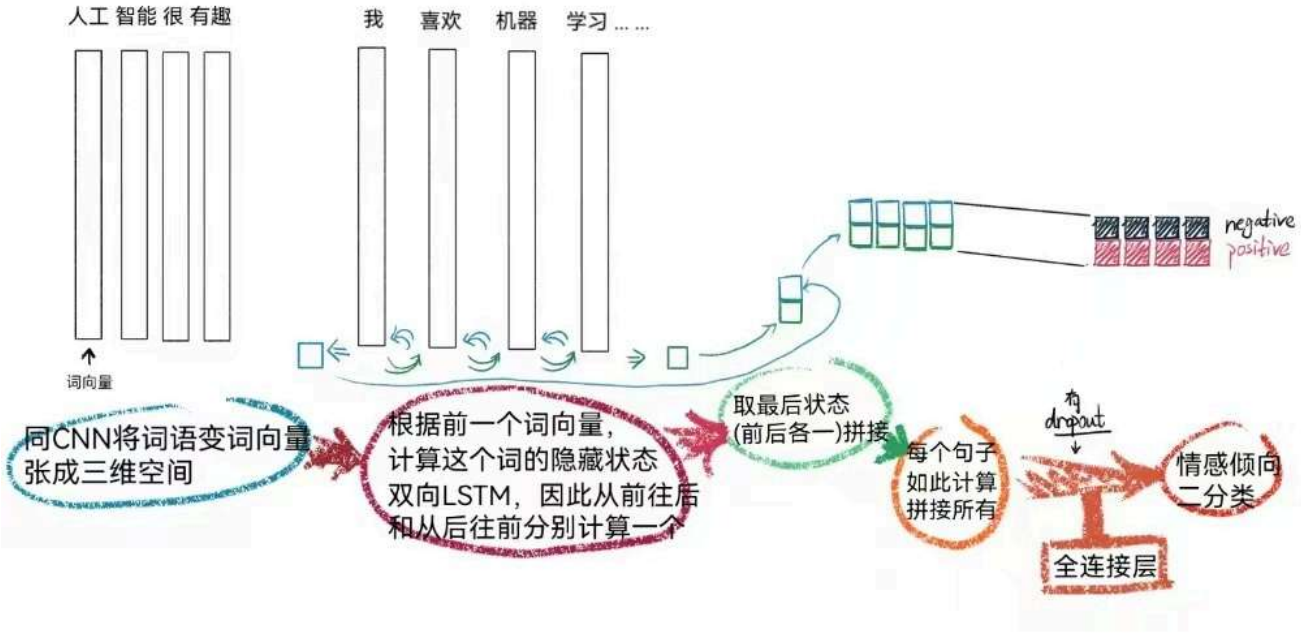
1. 模型结构图和流程分析

结构图

CNN 结构图



LSTM 结构图



流程分析

CNN 模型流程分析

1. 输入处理：
   - 输入已经分词的一批句子，根据word2idx文件中将分词的句子的每个词转换成词向量，由此张成 `[B, T, D]` 形状的三维空间。
   - 其中 `B` 是 batch size， `T` 是最大句子长度， `D` 是词向量维度。
   - 这是嵌入层embedding。
2. 维度变换：
   - 对词序列做一维卷积，因此需要将“词语”的维度移到最后，以适配 `nn.Conv1d` 。
   - 使用 `permute(0, 2, 1)` 将其变为 `[B, D, T]` ，
     以适配卷积操作（以词序为横轴，每个通道是一个 embedding 维度）。
3. 卷积操作：
   - 使用多个不同大小的卷积核（如2, 3, 4）对输入进行 1D 卷积，
     提取不同 n-gram 区间的局部特征。
4. 激活函数，池化：
   - 每个卷积输出经过 ReLU 激活函数后使用最大池化压缩为一维特征向量。
5. 特征拼接：
   - 将所有卷积核的池化输出拼接成一个长向量 `[B, num_filters × len(filter_sizes)]` 。
6. Dropout：
   - 对拼接后的向量进行 dropout 防止过拟合。
7. 全连接输出：
   - 通过一个全连接层层输出二分类的概率，分别对应正面/负面分类。

BiLSTM 模型流程分析

1. 输入处理：
   - 同CNN模型，句子经过词向量映射后，得到 `[B, T, D]` 的张量，
     表示每个句子中每个词的嵌入表示。
2. 双向 LSTM：
   - 输入喂入 `bidirectional=True` 的 LSTM 层，分别从前向和后向读取文本。
   - 使用默认激活函数 `sigmoid` 和 `tanh` 。
   - 得到隐藏状态张量 `h_n` ，形状为 `[2, B, H]` ，其中 `H` 为单向隐藏维度。
3. 拼接隐状态：
   - 取正向和反向两个最后时间步的状态 `h_n[-2]` 和 `h_n[-1]` ，
     并在维度上拼接得到 `[B, 2H]` ，作为整句话的语义表示。
4. Dropout：
   - 对拼接的 `[B, 2H]` 加入 Dropout 防止过拟合。
5. 全连接输出：
   - 同CNN，经过该层输出分类结果。

2. 实验结果：准确率与 F-score 指标等

本实验对比了基于双向 CNN 与 LSTM 的两种模型在情感分类任务上的性能表现，评估指标包括准确率（Accuracy）、F1-score、Precision 和 Recall。模型均在相同的预处理与词向量初始化条件下进行训练与测试。

模型类型	准确率 Accuracy	F1-score	Precision	Recall
CNN	**0.8455**	**0.8438**	**0.8652**	**0.8235**

模型类型	准确率 Accuracy	F1-score	Precision	Recall
BiLSTM	0.8157	0.8068	0.8606	0.7594

从结果可以看出，从结果可以看出，CNN 模型在所有评估指标上均优于 BiLSTM，不仅在准确率与 F1-score 上更高，同时在 Precision 与 Recall 上也表现更为出色。这说明 CNN 模型在本任务中具有更强的整体判别能力，既能准确识别正例，又能尽可能多地覆盖正样本。

- **Accuracy（准确率）**：整体分类正确的样本比例，反映模型的整体判断能力。
- **Precision（精确率）**：在模型预测为正的样本中，有多少是真的正样本，代表模型输出的“可信度”。
- **Recall（召回率）**：在所有实际为正的样本中，有多少被模型正确预测，代表模型的“覆盖能力”。
- **F1-score**：精确率和召回率的调和平均，综合反映模型对正类的整体表现，尤其适用于类别不平衡的任务。

这可能反映出，在本数据集中，局部特征（如 n-gram 模式）比长距离依赖结构更为关键，使得基于卷积操作的模型在提取有效语义信息时更具优势，而 BiLSTM 虽具备建模上下文的能力，但在实际表现上略显不足。

3. 不同参数的效果比较与分析

CNN模型

对四个主要超参数进行了网格搜索调参，分别为：

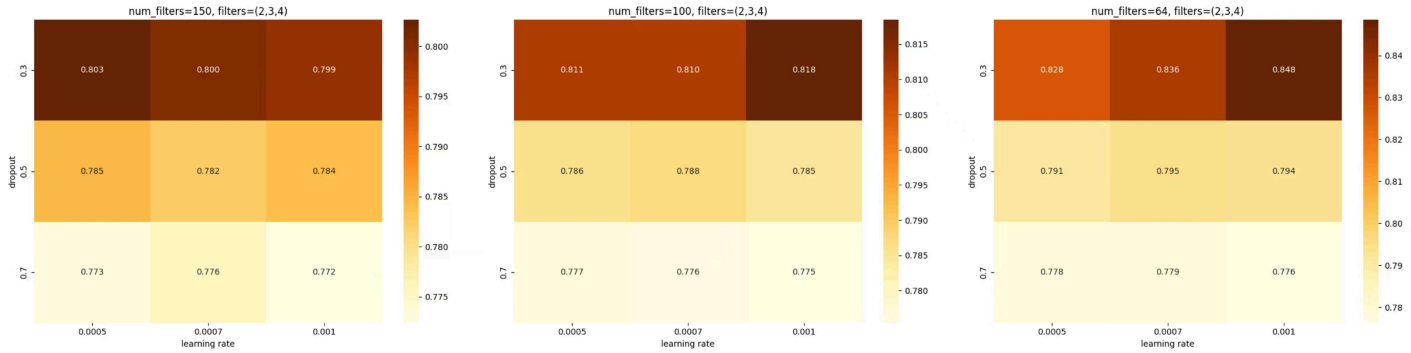
- **filters**：卷积核的大小组合（如 (2, 3, 4)），表示模型尝试从不同长度的 n-gram 局部模式中提取特征。较小的卷积核更关注小范围的词语之间关系，较大的卷积核能捕捉更长的上下文结构。
- **dropout**：丢弃率，用于防止模型过拟合。在训练时会随机将部分神经元输出置零，迫使模型不能过分依赖某些特征。值越大，每次训练时会随机“丢弃”更多的神经元输出，可能损失有用信息。
- **num_filters**：每种卷积核大小对应的卷积核数量，决定了模型提取多少种不同的局部特征。数量越多，特征表达越丰富，但模型参数也越多。
- **lr（learning rate）**：学习率，控制模型在每一步参数更新中的步长。过高的学习率会导致训练震荡甚至发散，过低则可能训练过慢、陷入局部最优。

实验结果：

- **Dropout Rate** 较低（如 0.3）时模型表现更好，说明本任务中模型未出现严重过拟合，较高的 Dropout 反而可能导致信息丢失。
- **卷积核数量（num_filters）** 趋于适中（如 64）时，准确率更高。在此基础上增加通道数量反而造成准确率降低。说明卷积核数量多时，虽然能带来更强的特征提取能力，但也引入了更多噪声和过拟合风险。
- **Filter Sizes** 为 (2, 3, 4) 的组合整体效果最佳，说明本数据集中包含较多可通过短语级模式（n-gram）捕捉的情感线索。
- **学习率（learning rate）** 的最佳值在 `0.001` 附近，较小的学习率如 `0.0005` 会导致训练过慢，较大的学习率如 `0.01` 则可能使模型发散。

综上，最佳配置 `(2,3,4) + dropout=0.3 + num_filters=64 + lr=0.001` 验证集准确率最高，为 `0.8485` 。

同时注意到，对于不同的卷积核数量，学习率和 Dropout Rate 的最优方案不同。由于LSTM模型中也对这两个超参数进行了调整，将热图绘制如下进行可视化和比较：



LSTM模型

对三个主要超参数进行了网格搜索调参，分别为：

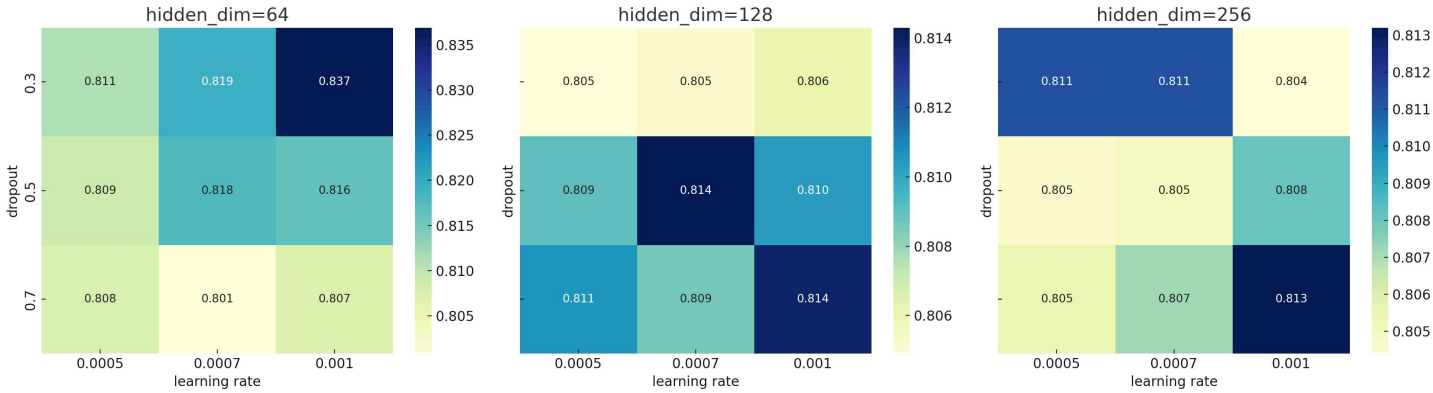
- **hidden_dim**：LSTM 的隐藏状态维度，决定模型每个时间步能记忆的特征容量。维度越大，模型的记忆能力越强，但计算成本和过拟合风险也随之增加。
- **dropout**：同CNN，丢弃率。
- **learning rate**：同CNN，学习率。

实验结果：

- **hidden_dim=64** 整体表现最佳，说明较小容量的 LSTM 就已能捕捉主要情感线索，更大的隐藏维度反而因过拟合表现更差。
- **Dropout=0.3** 在hidden_dim=64中表现更优（虽然在hidden_dim=128和hidden_dim=256时，更高的dropout rate更好，可能跟隐藏状态维度增加导致一定的“过拟合”有关，此时增加屏蔽能增加模型的泛化能力），说明此任务中适度正则化比过强的随机屏蔽效果更好。
- **Learning rate=0.001** 整体表现最为稳定。

综上，最佳配置 `hidden_dim=64 + dropout=0.3 + lr=0.001` 验证集准确率最高，为 `0.8369` 。

热图绘制如下：



4. Baseline 模型与 CNN、RNN 的对比分析

在本次实验中，MLP 被作为 Baseline 模型进行训练与评估。虽然其结构相较于 CNN 和 BiLSTM 更为简单，仅使用了平均池化后的全连接层进行分类，但在实际效果上却表现出了极强的竞争力。

MLP模型

1. **Embedding**
   同CNN和LSTM，将分词后的句子的每个词转化为词向量
2. **平均池化**（Mean Pooling）
   对句子的每个词向量在维度T进行平均，即求句子所有词向量的均值 `mean` ，得到该句子整体的向量（将一个 `[T, D]` 的矩阵压缩为一个 `[D]` 的向量）。

```
x = self.embedding(x)          # [B, T, D]
x = x.mean(dim=1)              # → [B, D]
```

3. **Dropout**：
   同CNN和LSTM，防止过拟合。
4. **全连接输出**：
   同CNN和LSTM，输出二分类结果。

最终的测试集结果如下：

模型	Accuracy	F1-score	Precision	Recall
MLP	**0.8591**	**0.8571**	**0.8814**	**0.8342**
CNN	0.8455	0.8438	0.8652	0.8235
BiLSTM	0.8157	0.8068	0.8606	0.7594

从表中可以看出，MLP 在所有四项指标上均超过了 CNN 和 LSTM，尤其在 Precision 和 F1 上拉开了明显差距。这说明：

- MLP 模型虽然结构简单，但由于使用了预训练词向量，使用简单的平均即可有效捕捉全局语义特征，且能减少过拟合的风险。
- 相比之下，BiLSTM 更依赖序列上下文建模，对长距离依赖较敏感，可能未充分学习到有效特征。CNN 虽然通过局部 n-gram 特征提取，表现比LSTM更优，但仍不如 MLP 的全局平均特征在本任务上有效。
- 从CNN模型比BiLSTM的方法更优方面来看，本次情感二分类任务并不怎么依赖“上下文”的长段语义结构（LSTM的优势未能发挥），而是更依赖于“局部”的结构甚至“单个词语”的模式，最优CNN所用的（2，3，4）卷积核的大小也并不大。MLP 虽然没有提取上下文的信息，只做了简单平均，但已经足以完成本次二分类任务。
- 这可能是合理的，推测本次文本情感分类由于使用了预训练词向量，且使用的文本较少隐喻、反讽等依赖上下文的表达，多数是表达直白、直接使用负面词语的“评论”，因此其中一两个能显著拉低“句平均词向量”的负面词语即可做出情感二分类的判断。此时再引入上下文依赖的复杂模型反而可能因为过拟合降低准确率等评价指标。

5. 问题思考

1. 实验训练什么时候停止是最合适的？

本实验采用固定轮数训练（Epochs = 10），每轮训练后在验证集上评估模型性能，若当前轮的准确率超过历史最高值，则保存当前模型，从而保证保存的是这10轮训练中的最优模型。

固定迭代次数

- 优点：实现简单，适合实验流程清晰的场景，不依赖外部验证机制。
- 缺点：无法动态适应训练状态。可能未训练到最优就过早停止；或即使已经达到了最优仍然继续训练，导致训练时间较长。

更进一步的方法可以引入 **早停（early stopping）**，当验证集性能连续若干轮不再提升时提前终止训练，以节省时间并避免过拟合。

通过验证集调整

- 优点：能更灵活地根据实际学习情况做出判断，有利于缓解过拟合。
- 缺点：实现更复杂；模型性能对验证集划分较为敏感，若验证集代表性不足，可能导致早停点不合理，影响最终模型效果。

2. 实验参数的初始化是怎么做的？

实验参数

本实验使用的是预训练词向量作为嵌入层参数初始化，对于其他模型层参数则采用 PyTorch 默认方式，未显式指定初始化函数。

- Embedding 层：
  - 手动初始化。将填充词向量的 PAD 初始化为 0 ，为 `word2idx` 里面没有对应的词UNK分配了均值为0的正态分布随机向量（标准差默认为 1），其余词使用预训练的 `word2idx` 词向量进行初始化。
- CNN 模型中的卷积层 Conv1d 和 线性层 Linear：
  - `nn.Conv1d` 和 `nn.Linear` 默认调用 `kaiming_uniform_`，使用 Kaiming（He）初始化。
  - 根据上一层的输入节点数 `fan_in` 自动设置权重初始化范围，使每层输出的方差尽量保持不变，适用于 ReLU 激活函数。
  - Normal 初始化，如下公式：

$$w \sim \mathcal{N}(0, \sqrt{\frac{2}{\text{fan_in}}})$$

  - 用于卷积层的 Uniform 初始化则用如下公式：

$$w \sim \mathcal{U}\left(-\sqrt{\frac{6}{\text{fan_in}}}, \sqrt{\frac{6}{\text{fan_in}}}\right)$$

- LSTM 模型默认使用均匀分布初始化（Uniform）
  - 在 $[-k, k]$ 范围内均匀采样，其中 $k = \sqrt{1/\text{hidden_size}}$。
  - 偏置项默认初始化为 0。

初始化方法及其适用场景：

1. 零均值初始化

- 初始化时使权重的均值为 0。
- 避免模型在初始阶段对某一方向产生偏置。
- 通常分布取高斯分布或均匀分布初始化，初始化的权重 $W \sim \mathcal{N}(0, \sigma^2)$ 或 $W \sim U(-a, a)$，期望为 0。
- 适用场景：Kaiming、Xavier 等初始化方法都基于零均值的假设，适用于绝大多数神经网络。

2. 高斯分布初始化

- 从正态分布中采样初始权重 $W \sim \mathcal{N}(0, \sigma^2)$。
- 优点：易于实现，参数空间连续平滑。
- 缺点：若标准差过大，可能导致梯度爆炸；若太小，则可能梯度消失。
- 适用场景：小型或浅层网络，激活函数为 sigmoid / tanh 时更敏感，不推荐用于深层网络或 ReLU 网络。

3. 正交初始化

- 初始化时生成一个正交矩阵 Q，使 $Q^\top Q = I$, 使用这个矩阵为权重矩阵初始化赋值。
- 它能保持向量在前向和反向传播中的范数不变。
- 先生成一个随机矩阵，再使用 QR 分解获取正交矩阵。
- 适用场景：RNN， LSTM， GRU 等循环神经网络，可有效缓解梯度消失问题，提升模型对“前面状态”的“保持信息”的能力。
- 对于每一层参数相同的权重共享结构，以及输入有序列性的建模（如本次实验的文本分析），能帮助网络更好地传播梯度。

3. 过拟合是深度学习常见的问题，有什么方法可以防止训练过程陷入过拟合？

模型在训练集上学习得过于“完美”而导致泛化能力下降，即为“过拟合”。
常见的防过拟合策略包括：

- Dropout：在训练过程中随机丢弃部分神经元输出，强迫模型依赖更少的“神经元”输出的特征进行学习。
- L2 正则化（权重衰减）：在损失函数中对模型参数权重值太大进行惩罚，限制权重值，避免模型过度拟合。
- 数据增强：如同义词替换，增加语义不变但表达不同的训练样本；再如遮蔽，随机用符号替换部分词。
- 增加训练数据：本质上来说，过拟合是因为模型参数远多于有效样本；从更多的训练数据，模型会获得更全面的输入分布，从而改善过拟合。

- 使用验证集，早停：在验证集性能不再提升时提前终止训练，或根据验证集保存准确率最好的模型（不一定是最后那轮训练的模型）。

本实验使用了 Dropout 和验证集模型选择（虽然没有引入“早停”机制，但是根据验证集结果保存最优模型），能在一定程度上改善过拟合问题。

4. 试分析 CNN，RNN，MLP 三者的优缺点

模型	优点	缺点
CNN	擅长捕捉局部 n-gram 特征，结构简单，训练速度快。	难以捕捉长距离依赖， 固定感受野可能限制全局语义理解。
RNN	能自然建模序列关系，特别适合表示时序结构 （如上下文依赖）。	训练时间长，容易出现梯度消失问题； 长句建模效果有限。
MLP	结构最简单，训练快，适合作为 baseline。	无法捕捉词序信息，只能依赖词向量平均， 特征提取能力弱。

在本实验中，尽管 MLP 是最简单的模型，但在数据集上表现出令人惊讶的性能（准确率 85.91%），说明该任务可能对词序依赖要求不高，而整个句子的平均表示已经足够分辨情感极性。

6. 心得体会

心得体会

在本次情感分类实验中，我最初尝试的是 CNN，通过不同大小的卷积核提取 n-gram 局部特征，在多组参数组合下表现稳定，训练也较为快速。

随后，我实现了单向 LSTM 模型。然而，训练过程中准确率始终维持在约 **50%** 左右，等于随机猜测，显然模型并未有效学习。经过搜索相关资料后，我意识到单向 LSTM 只能从前往后处理文本，可能丢失关键信息。

于是我将其改为双向 LSTM（BiLSTM），使模型能同时从前后两个方向读取序列，训练结果也得到改善，准确率也和CNN一样达到了 80% 。可能因为在文本分类任务中，信息的前后依赖是对等且关键的，单向结构难以胜任这一任务。

但令我惊讶的是，当我实现了一个结构最简单的 MLP模型作为baseline模型时，它反而取得了三者中最高的准确率和 F1 值。通过查阅资料，我认为这是由于在任务较简单、数据量有限的情形下，复杂模型未必一定胜过简单模型。CNN 和 LSTM 根据前后文提取特征的能力固然更强，但也更容易过拟合；而 MLP 的参数更少，在情感倾向明显且不依赖于上下文的时候，反而更容易学习到关键特征。

因此，我认识到，模型不能一味追求复杂，而要根据任务需求、数据特性做出选择。同时，我也掌握了神经网络的构建流程、初始化与调参技巧，并初步了解了不同模型结构的优缺点。